Operating Systems

ASSIGNMENT 1

CO1: *Apply the concepts of operating system types, structures and system calls to demonstrate their functions in various computing environments – history of operating systems, mainframe, desktop, multiprocessor, distributed, clustered, real-time and handheld systems, OS structure, services, system calls, system design, CPU states, memory hierarchy and I/O systems.*

Assignment Title: Evaluate and Justify the Operating-System Design of a Smart Hospital (Total Marks - 50)

Nature of Assignment: Individual

Name:G.Krishitha(192524451)

Duration: 1 Week

Submission Date: 03.11.2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator). • Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted. • Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: https://github.com/krishithagopi07-cell/CSA0415-OS-ASSIGNMENT.git

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the

alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

SCENARIO AND DESIGN APPROACH

The hospital has 500 beds and requires bedside patient monitors, nurses' tablets, doctors' workstations, a highly available patient-record database, an MRI imaging server and secure communication with two partner hospitals. Bedside monitors must raise an alarm within 10 milliseconds after an abnormal reading. The system must also scale from one ward to the full hospital and then to three connected hospitals.

A single general-purpose operating system is not suitable for every component because the devices have different priorities. Patient monitors need predictable response, tablets need low power consumption, workstations need usability, and servers need high availability, throughput and security. Therefore, this report recommends a mixed operating-system design with standard interfaces and centrally managed security.

TASK 1 – EVALUATE THE REQUIREMENTS OF EACH COMPUTING ENVIRONMENT

The operating system must be selected according to the most important workload and risk of each environment. Patient safety and predictable alarm delivery receive the highest priority on bedside monitors. Server availability and data integrity are critical for the database.

Environment	Ranked requirements (highest first)	Justification
Bedside monitors	1. Deadline ≤10 ms; 2. Reliability; 3. Sensor I/O; 4. Security; 5. Low memory/power	A missed or late alarm can affect patient safety. Predictable scheduling is more important than general-purpose features.
Nurses' tablets	1. Security; 2. Battery life; 3. Reliability; 4. Responsive UI; 5. Connectivity	Portable devices must last a shift and protect patient information while supporting wireless access.
Doctors' workstations	1. Security; 2. Usability; 3. Responsiveness; 4. Compatibility; 5. Manageability	They run clinical applications, imaging viewers and communication tools for multiple users.
Patient-record database	1. Availability; 2. Data integrity; 3. Security; 4. Throughput; 5. Recovery	Records must remain accessible during maintenance and hardware failure. Transactions must be protected.
MRI imaging server	1. Processing throughput; 2. Memory capacity; 3. I/O bandwidth; 4. Reliability; 5. Security	Large images require parallel processing, fast storage and high-bandwidth data movement.
Partner-hospital link	1. Confidentiality; 2. Integrity; 3. Availability; 4. Interoperability; 5. Scalability	Records must be exchanged securely and consistently across separate networks and systems.

Conflicting requirements: a general-purpose scheduler may maximise average throughput but cannot necessarily guarantee a 10 ms worst-case response. Strong encryption and access checks consume CPU time, while battery-saving modes may delay background communication. High availability and redundancy increase cost and administrative complexity.

Patient-safety-critical requirements include bounded alarm latency, reliable sensor acquisition, correct timestamps, secure device identity and safe behaviour during network failure. Local alarms should continue even when the hospital network is unavailable.

Historical lessons: batch systems improved efficient processing but are unsuitable for immediate alarms; time-sharing supports many interactive users on workstations; personal-computer operating systems provide desktop

applications; mobile operating systems add power management and wireless support. Real-time and embedded operating systems are appropriate for monitors.

Alternative concept: one general-purpose OS everywhere

Using one desktop-style OS on all devices simplifies administration and staff training, but it does not guarantee bounded alarm latency. A non-real-time scheduler can experience an unbounded or difficult-to-prove delay under load, whereas a properly configured hard real-time design can provide a verified deadline bound. A full OS also requires more RAM, storage and background services than a small embedded OS. Centralised administration may be simpler, but patching and configuration must still be tailored to each device. The single-OS approach is therefore rejected for safety-critical monitors.

Complexity: choosing a different OS does not automatically change an algorithm's Big-O complexity; it changes constant overhead, resource requirements and timing guarantees. Alarm processing should be constant-time or tightly bounded per sample, O(1), with fixed memory use O(1) for a fixed-size sample buffer.

TASK 2 – JUDGE THE TYPE OF OPERATING SYSTEM FOR EACH ENVIRONMENT

Environment	Recommended OS type	Reason and rejected alternatives
Bedside monitors	Embedded hard real-time OS	Deadline-sensitive sensor and alarm tasks. Desktop/time-sharing OS does not inherently guarantee deadlines.
Nurses' tablets	Handheld/mobile OS	Power management, touch interface, wireless networking and device security. Desktop OS is generally heavier.
Doctors' workstations	Desktop OS	Interactive multi-application environment with broad peripheral and software support.
Patient-record database	Clustered server OS	Fail-over between nodes improves service continuity. One large standalone machine remains a single point of failure.
MRI imaging server	Multiprocessor server OS	Parallel image processing and large memory capacity. A single-processor system limits throughput.
Partner-hospital link	Distributed-system architecture	Separate hospitals exchange data over a network while maintaining local services. A single central computer would create dependency and network bottlenecks.

Mainframe systems are designed for very large central workloads and transaction volumes; they may be useful for enterprise-scale records but are not required for every endpoint. Desktop systems suit interactive workstations. Multiprocessor systems use multiple CPUs or cores. Distributed systems coordinate independent networked computers. Clustered systems provide coordinated service and fail-over. Hard real-time systems require strict deadlines; soft real-time systems tolerate occasional delay with degraded quality. Handheld systems prioritise mobility and energy efficiency.

SMP versus AMP for MRI

Symmetric multiprocessing (SMP) allows processors to share memory and operating-system responsibilities. It is the preferred general choice for the MRI server because the OS can schedule parallel image-processing tasks across available cores. Asymmetric multiprocessing (AMP) assigns specific tasks to particular processors and can offer predictable roles in specialised systems, but it is less flexible for a changing imaging workload.

Database cluster versus one large multiprocessor

A cluster of smaller servers with replicated data and automatic fail-over avoids dependence on one machine. If one node fails, another can take over, potentially within seconds depending on health checks and recovery configuration. A single large multiprocessor server may offer strong throughput and shared memory, but a motherboard, power or OS failure can stop the entire service. Clusters cost more to deploy and administer and require careful replication and split-brain protection. For a fixed machine, processor throughput may increase with core count but is limited by serial work, memory bandwidth and contention; memory capacity is determined by installed RAM and is not automatically multiplied by processor count.

TASK 3 – EVALUATE OPERATING-SYSTEM STRUCTURES

Structure	Reliability	Performance / updates	Hospital assessment
Monolithic (simple)	A kernel fault can affect the whole OS	Fast internal calls; tightly coupled changes	Good performance, but less fault isolation.
Layered	Clear separation can simplify verification	Layer crossings may add overhead; layers aid maintenance	Useful design discipline for complex systems.
Microkernel	Small kernel; many services run separately, improving isolation	Message passing may add overhead; services can be restarted separately	Strong choice for bedside monitors where fault containment matters.
Modular	Modules can be loaded/unloaded; core remains integrated	Near-monolithic performance; modules ease driver updates	Suitable for Linux server systems.
Hybrid	Combines approaches to balance performance and modularity	Can be efficient but internally complex	Common in commercial desktop/mobile systems.

Linux is generally a modular monolithic kernel with loadable kernel modules. Windows uses a hybrid kernel architecture. Android uses the Linux kernel with Android-specific framework and system services. FreeRTOS is a small real-time kernel; QNX is a microkernel-based real-time operating system.

Recommended structures: bedside monitors should use a small microkernel or safety-oriented RTOS with only essential drivers and services. The database server can use a mature modular Linux kernel because it needs high throughput, broad storage/network support and maintainable drivers. The database must be protected by clustering, monitoring and tested recovery procedures; kernel structure alone does not guarantee availability.

Alternative: monolithic kernel on bedside monitors

A monolithic design can service system calls with lower internal communication overhead because many services execute in kernel space. A microkernel may require message passing between user-space servers, adding context switches and communication cost. However, the microkernel can keep the kernel smaller and isolate faulty services. In a monolithic kernel, a defective driver can corrupt kernel memory and crash the complete device. For a monitor, the reliability benefit of isolation outweighs a modest and measurable communication cost, provided the selected RTOS meets the 10 ms deadline.

TASK 4 – EVALUATE THE SERVICES AN OPERATING SYSTEM MUST PROVIDE

Service	Monitors	Tablets / workstations	Database server
User interface	Minimal local status/alarm display	Full touch or desktop UI	Administrative console only
Program execution	Fixed verified tasks	Clinical apps and services	Database engine and support tools
I/O operations	Sensor, display and alarm I/O	Touch, Wi-Fi, storage and peripherals	Disk, network and backup I/O

File system	Small configuration/log storage	Managed local app data	Journalled, monitored data volumes
Communication	Authenticated local/network messages	Secure hospital network	Encrypted client and replication links
Error detection	Watchdog, sensor and task checks	OS/app diagnostics	Hardware, storage and database monitoring
Resource allocation	Bounded CPU and memory	Power-aware scheduling	CPU, memory and I/O scheduling
Accounting	Event and alarm logs	Audit and device management	Usage and transaction logs
Protection/security	Secure boot, least privilege	Encryption, screen lock, app controls	Strong authentication, roles, audit, encryption

For a bedside monitor, remove unnecessary general-purpose services such as a full desktop environment, consumer applications, printing, broad package managers and nonessential background daemons. Keep only sensor acquisition, alarm handling, watchdog, secure communication, timekeeping, minimal logging and update/recovery functions. This reduces attack surface and resource use.

The database server requires process and memory protection, authentication, role-based authorisation, transaction and file-system integrity, audit logging, encryption, network controls, backup, monitoring and resource allocation. Patient records should be separated by authenticated user identity and database permissions; one user's program must not access another patient's records without authorisation.

Alternative: full desktop OS on tablets

A full desktop OS brings additional background services, larger storage and memory requirements, longer boot/update times and more frequent CPU activity. Those costs can reduce battery life and leave less space for clinical applications. A managed handheld OS is preferred because it includes mobile power management and a controlled application environment. Exact memory, boot-time and battery differences depend on the selected device and must be measured on the actual hardware.

TASK 5 – EXAMINE SYSTEM CALLS

A system call is the controlled interface through which a user program requests a service from the operating-system kernel. Major categories are process control, file management, device management, information maintenance, communication and protection.

What happens during read()

1. The application calls the C library read() wrapper with a file descriptor, buffer address and byte count.
2. The wrapper places the system-call number and arguments in the platform-defined registers or memory locations.
3. A trap or system-call instruction transfers execution to kernel mode.
4. The kernel validates the descriptor, permissions and buffer range, then requests data from the file or device subsystem.
5. The kernel copies or maps available data to the user buffer and determines the number of bytes read.
6. The kernel returns a result; the processor switches back to user mode and the library wrapper returns to the application.

Parameters can be passed in CPU registers, through a memory block whose address is passed in a register, or through the stack, depending on the processor ABI. The exact convention is architecture-dependent.

The strace utility records system calls and their return values. For example, `strace -e trace=openat,read,write,close ./program` shows file-related calls made by a Linux program.

Alternative: stdio functions versus direct system calls

fopen(), fread() and fwrite() are C standard-library functions. They usually maintain a user-space buffer and issue fewer underlying read() or write() system calls. With a library buffer of approximately 4096 bytes, a file of n bytes may need roughly ceil(n/4096) data-transfer calls, plus open/close and other calls. Direct calls with buffer size b may require roughly ceil(n/b) reads and a similar number of writes. Library buffering consumes extra user-space memory, typically one or more buffers per open stream, but reduces expensive kernel transitions. Exact counts depend on buffering mode, file size and implementation.

TASK 6 – EVALUATE THE COST OF SYSTEM CALLS

An ordinary function call generally changes control flow within the same privilege level. A system call must cross the user/kernel boundary, validate arguments and permissions, execute kernel code and return safely. It may also cause scheduling or device I/O, so its cost is substantially higher than a simple function call.

For a file of n bytes, the approximate number of read calls with buffer size b is ceil(n/b), plus one final read that returns end-of-file for a loop that reads until EOF. Larger buffers reduce call count but use more memory. Very large buffers may provide diminishing returns and can increase memory pressure.

Completed C program – read() and write()

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
#include <stdlib.h>

long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    if (bufsize == 0 || bufsize > sizeof(buf)) return -1;

    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }

    long calls = 0;
    ssize_t n;
    while (1) {
        do { n = read(in, buf, bufsize); } while (n < 0 && errno == EINTR);
        calls++;
        if (n == 0) break;
        if (n < 0) { calls = -1; break; }

        ssize_t written = 0;
        while (written < n) {
            ssize_t w = write(out, buf + written, (size_t)(n - written));
            if (w < 0 && errno == EINTR) continue;
            if (w <= 0) { calls = -1; break; }
            written += w;
        }
        if (calls < 0) break;
    }
    close(in);
    close(out);
```

```
    return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w");
    if (!f) return 1;
    for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
    fclose(f);

    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\\n",
               sizes[i], copy_file("records.dat", "copy.dat", sizes[i]));
    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

Compile and run on Linux:

```
gcc -std=c11 -Wall -Wextra -O2 copy.c -o copy
./copy
strace -c -e trace=read,write,openat,close ./copy
```

Expected read-call counts for a 10,000-byte file are 10,001 with a 1-byte buffer, 21 with a 512-byte buffer, and 4 with a 4096-byte buffer. The extra call is the final EOF read. The provided expected output is a reference; compare it with your own execution and save the terminal output.

Buffer size	Expected read() calls	Interpretation
1 byte	10001	Very high system-call overhead.
512 bytes	21	Much fewer calls.
4096 bytes	4	Low call count for this test file.

Alternative: memory-mapped files (mmap)

With mmap(), a process maps file contents into its virtual address space and accesses them through memory operations. Mapping and unmapping require relatively few system calls compared with repeatedly reading small blocks. It can avoid an explicit user-buffer copy in some paths, but page faults still occur and the kernel manages the file-backed pages. Mapping a very large file can consume a large virtual address range and page-cache memory; it does not mean the entire file is physically loaded at once. For a sequential backup tool, buffered read/write is simple and predictable; mmap may be useful after benchmarking.

TASK 7 – EVALUATE PROCESS CONTROL SYSTEM CALLS

fork() creates a child process that initially has a logical copy of the parent's address space, commonly implemented using copy-on-write. exec() replaces the current process image with a new program. wait() or waitpid() collects a child's termination status. exit() terminates a process and makes its status available to the parent.

A shell parses a command, creates a child with fork(), runs the requested program using an exec-family call, and waits for completion when the command is foreground. The shell obtains the exit status through waitpid(). A zombie is a terminated child whose status has not yet been collected. An orphan is a child whose parent terminates first; it is adopted by a system process that can reap it.

Completed C program – fork(), execlp() and waitpid()

```
#define _POSIX_C_SOURCE 200809L
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid;
    int status;

    printf("shell: running job 1\n");
    fflush(stdout);

    pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        execlp("expr", "expr", "6", "*", "7", (char *)NULL);
        perror("execlp");
        _exit(127);
    }
    if (waitpid(pid, &status, 0) < 0) { perror("waitpid"); return 1; }
    if (WIFEXITED(status))
        printf("job 1 exit status %d\n", WEXITSTATUS(status));

    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) _exit(3);
    if (waitpid(pid, &status, 0) < 0) { perror("waitpid"); return 1; }
    if (WIFEXITED(status))
        printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

Compile and run:

```
gcc -std=c11 -Wall -Wextra process_demo.c -o process_demo
./process_demo
```

Expected output:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

The first fflush(stdout) ensures the message is written before fork(), so the child does not inherit a copy of an unflushed output buffer. If the second fflush() is removed, output is redirected to a file, and the child calls exit(3), the buffered message may be flushed by both parent and child, causing the line to appear twice. _exit() terminates without flushing inherited C library buffers.

Alternative: posix_spawn() or threads

posix_spawn() combines process creation and program execution through a library interface and can be more efficient on some systems, especially where a child does not need complex setup between fork and exec. Threads share the same address space and many process resources, so they can be cheaper to create, but they are not isolated: a faulty command can corrupt shared memory. Separate processes provide stronger address-space isolation. For 1,000 jobs, measure elapsed time and peak memory on the same machine; actual results depend on OS, workload and resource limits.

TASK 8 – EVALUATE CPU STATES AND PROTECTION

Modern processors support at least user mode and kernel mode. User programs run with restricted privileges. The mode bit and processor privilege levels determine whether an instruction is permitted. Privileged operations, such as changing interrupt control or page-table configuration, are restricted to the kernel.

An interrupt is an asynchronous signal from hardware; a trap or exception is a synchronous event caused by an instruction or fault. A timer interrupt allows the OS to regain control and schedule another task. If ordinary monitor software could disable interrupts, it could prevent timely handling of sensor events and OS scheduling, threatening the 10 ms alarm deadline.

Memory protection uses virtual address spaces and hardware page permissions. A user process cannot normally read another process's memory because the MMU checks access permissions. Kernel mode is reserved for trusted OS code, while applications use validated system calls.

Alternative: polling instead of interrupts

Polling checks a sensor repeatedly. If the polling interval is P, the detection delay can approach P, plus processing time; average delay is approximately P/2 for uniformly timed events. Interrupt-driven input can notify the CPU soon after an event, with latency determined by interrupt masking, priority and handler execution. Polling can waste CPU and battery when no event occurs. A carefully designed hybrid may poll at a controlled rate for some devices, but the monitor's deadline must be verified under worst-case conditions.

TASK 9 – EVALUATE THE MEMORY HIERARCHY AND I/O SYSTEMS

The memory hierarchy balances speed, capacity and cost. Faster levels are smaller and more expensive per byte; slower levels provide larger persistent capacity.

Level	Typical access time (approximate)	Typical capacity / role
CPU registers	Sub-nanosecond to a few cycles	Bytes to a few KB per core; immediate operands
CPU cache	About 1–20 ns depending on level	KB to tens of MB; frequently used data
Main memory (RAM)	About 50–100 ns	GBs; active programs and working data
NVMe / SSD	Tens to hundreds of µs	Hundreds of GB to TB; fast persistent storage
Hard disk	Several ms	TBs; lower-cost bulk storage
Network storage	Variable: sub-ms to many ms or more	Shared scalable storage; depends on network and service

Patient records belong in a highly available database backed by durable SSD or enterprise storage with replicated backups. MRI images require high-capacity, high-throughput storage, often a fast local cache plus managed long-term storage. Recent monitor readings may be buffered locally for immediate operation and transmitted to a central time-series or clinical data service; loss of network connectivity must not stop local alarms.

Programmed I/O makes the CPU repeatedly check device status and move data. Interrupt-driven I/O lets the device notify the CPU when service is needed. Direct Memory Access (DMA) transfers blocks between a device and memory with limited CPU involvement. DMA is appropriate for large MRI image transfers because it avoids per-word CPU handling and reduces interrupt overhead. Caching speeds record lookup by keeping frequently accessed indexes and records in faster memory, while requiring consistency and access-control safeguards.

Alternative: interrupt per word instead of DMA

For a 500 MB image, the number of interrupts would depend on the transfer word size. At 4 bytes per word, 500,000,000 / 4 = 125,000,000 interrupts (using decimal MB); at 8 bytes it would be 62,500,000. Handling this many interrupts would consume substantial CPU time. DMA can transfer a large block using a small number of setup/completion events. DMA requires appropriately sized buffers and careful cache coherency; interrupt-per-word handling avoids a large DMA transfer buffer but creates enormous CPU overhead. In practice, DMA with bounded buffers is the preferred method.

TASK 10 – MEASURE AND COMPARE

Measurements must be collected on the Linux system actually used for the assignment. Results depend on processor, memory, kernel, compiler, storage and background activity. The following commands provide repeatable evidence; insert your own outputs in the tables.

A. System-call counts and copy time

```
strace -c -e trace=read,write ./copy
/usr/bin/time -v ./copy
```

Buffer size	read() calls from program	strace read() count	User time	System time	Elapsed time
1 byte	[enter]	[enter]	[enter]	[enter]	[enter]
512 bytes	[enter]	[enter]	[enter]	[enter]	[enter]
4096 bytes	[enter]	[enter]	[enter]	[enter]	[enter]

The program's counter includes read() calls made by copy_file(). strace may also show reads performed by the C runtime or dynamic loader unless the trace is restricted to the relevant process and syscall. Explain any difference between the program counter and strace summary.

B. Process versus thread creation

Create a benchmark that loops 1,000 times, creating and reaping a child process each time with fork()/waitpid(). Create a second benchmark that creates 1,000 threads with pthread_create() and joins them with pthread_join(). Use CLOCK_MONOTONIC for elapsed time and compile the thread program with -pthread.

```
gcc -std=c11 -O2 process_bench.c -o process_bench
./process_bench

gcc -std=c11 -O2 -pthread thread_bench.c -o thread_bench
./thread_bench
```

Operation	Number	Elapsed time	Peak memory / notes
fork() + waitpid()	1000	[your measurement]	Separate process address spaces; copy-on-write
pthread_create() + join()	1000	[your measurement]	Shared address space; per-thread stack and metadata

Threads are often faster and use less total memory than separate processes because they share code, data and many resources. Each thread still needs a stack and kernel/runtime bookkeeping. Processes provide stronger isolation. Report the actual measured values and repeat each test several times, using the median to reduce noise.

C. Interpretation

Compare measurements with the predictions: increasing the copy buffer should reduce system-call count; elapsed time may stop improving once storage/cache throughput becomes the bottleneck. Thread creation is

often cheaper than process creation, but the exact gap is system-dependent. State whether the results support your earlier design choices and explain unexpected results.

TASK 11 – RECOMMEND AND DEFEND THE OPERATING-SYSTEM PLAN

Component	OS type / structure	Services and protection	I/O and availability
Bedside monitors	Embedded hard RTOS; small microkernel or safety-oriented RTOS	Minimal tasks, watchdog, secure boot, least privilege, signed updates	Interrupt-driven sensor input; local alarm path; bounded scheduling
Nurses' tablets	Handheld/mobile OS	Encryption, screen lock, managed apps, authentication, remote wipe	Wireless communication; local secure cache; power-aware I/O
Doctors' workstations	Desktop OS; modular/hybrid kernel	User accounts, endpoint protection, patching, role-based access	Standard drivers, secure network access, managed storage
Patient database	Server OS; modular kernel; clustered deployment	Database access control, audit, encryption, backups, replication	Redundant network/storage; health checks and automatic fail-over
MRI imaging	Multiprocessor server OS; SMP	Parallel processing, resource controls, restricted service accounts	High-throughput storage and DMA-capable device I/O
Partner hospitals	Distributed architecture across sites	Mutual authentication, encryption, authorisation, audit and data minimisation	Secure APIs/message exchange, retry queues and replicated services

This mixed plan matches each OS to its workload instead of forcing all devices into one design. The monitor's local safety function remains independent of database or internet availability. The clustered database reduces downtime, while the distributed architecture supports record exchange across three hospitals. The plan's limitations include higher licensing/support and integration cost, multiple patching processes, interoperability challenges, network outages and the need to validate every real-time deadline.

Alternative: virtualise the entire system on two large servers

Virtual machines can consolidate workstations and many server workloads, simplify deployment and improve hardware utilisation. However, hosting bedside-monitor control on general-purpose virtual machines introduces hypervisor scheduling, shared-resource contention and additional failure dependencies. A two-server failure domain may affect many services at once. Virtualisation also consumes memory and CPU for the hypervisor and guest operating systems. It is suitable for selected server workloads with redundancy, resource reservations and tested recovery, but not as a replacement for independent safety-critical monitor controllers.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN

A. Impact analysis

Remote home-monitoring devices add unreliable internet connectivity, different home network conditions, remote configuration and a larger attack surface. The original bedside real-time requirement remains relevant locally, but transmission to the hospital cannot be guaranteed within 10 ms over the public internet. Moving records to a cloud service changes the database deployment, identity management, data residency, backup, availability and partner-sharing arrangements.

- Affected OS types: embedded/mobile OS on home devices, server OS and distributed/cloud infrastructure for records.

- Affected services: secure networking, remote updates, buffering, time synchronisation, identity management, audit, backup and recovery.
- Affected protection: device authentication, encryption in transit and at rest, least privilege, secure boot, key management and monitoring.

B. Evaluation of the original design

The original local monitor and hospital server design remains suitable for in-hospital alarms because it keeps time-critical decisions close to the patient. It is not sufficient by itself for home devices and cloud-hosted records. Home connectivity can be interrupted or delayed, so devices must store readings temporarily and continue local alerts. Cloud services can improve geographic availability and scaling, but depend on provider controls, network connectivity, service-level agreements and secure configuration.

C. Adaptation / redesign

Decision: MODIFY the original design rather than replace it. Retain embedded real-time operating systems for local monitors, mobile/handheld operating systems for portable devices, desktop systems for clinical workstations, and a clustered server design for critical local services. Add the following:

7. Use an embedded real-time OS on home monitoring devices, with local threshold detection, safe alarm behaviour and encrypted store-and-forward buffering.
8. Use authenticated encrypted connections to a cloud-based records platform. Devices should have unique identities and certificates, and remote updates must be signed and verified.
9. Deploy records as a distributed service across availability zones or regions where appropriate, with replication, backups, monitoring and tested disaster recovery.
10. Use role-based access, strong authentication, audit trails, encryption at rest and in transit, and controlled APIs for partner hospitals.
11. Define data synchronisation rules, duplicate detection, timestamps and conflict handling so delayed readings are not mistaken for current measurements.

D. Evidence-based defence

Modification is the best decision because it preserves the verified low-latency local alarm path while adding cloud scale and remote access. The internet is not a deterministic real-time network, so cloud processing must not be placed in the critical alarm loop. The distributed cloud service improves sharing and can improve resilience, but increases dependence on network security, identity services and provider availability. The final architecture therefore separates safety-critical local control from non-critical remote storage and analytics. The choice should be validated through alarm-latency tests, network-loss tests, fail-over drills, penetration/security testing and measured recovery times.

CONCLUSION

A smart hospital requires a combination of operating-system types and structures. Hard real-time or embedded systems are appropriate for bedside and home monitors; handheld systems suit nurses' tablets; desktop systems suit doctors' workstations; clustered server systems protect patient records; and multiprocessor systems support MRI processing. A distributed architecture enables secure sharing across hospitals and cloud services. The design prioritises patient safety, availability, data integrity, privacy and scalability. Actual execution outputs and measurements must be added to demonstrate that the implementation meets the stated requirements.